

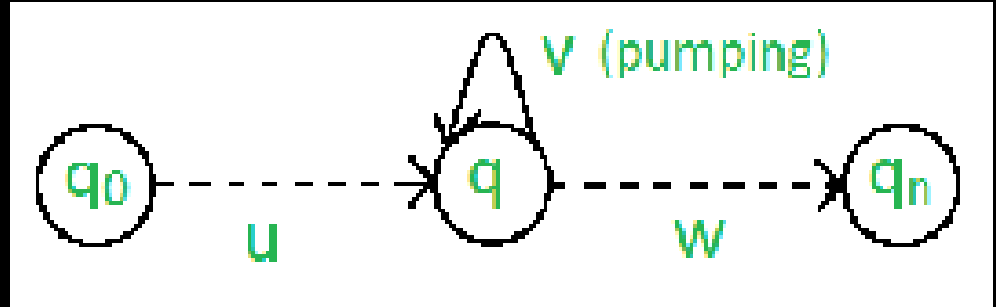
Pumping Lemma For Regular Languages

- Pumping Lemma is used as a proof for irregularity of a language. Thus, if a language is regular, it always satisfies pumping lemma. If there exists at least one string made from pumping which is not in L , then L is surely not regular.
- The opposite of this may not always be true. That is, if Pumping Lemma holds, it does not mean that the language is regular.
- Pumping Lemma is used to prove that some of the language is non-regular.
- For the pumping lemma, i/p is NRL & o/p is also NRL.



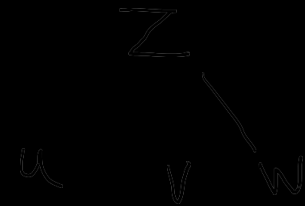
Pumping Lemma For Regular Languages

- For any regular language L , there exists an integer n , such that for all $z \in L$ with $|z| \geq n$, there exists $u, v, w \in \Sigma^*$, such that $z = uvw$, and
 - $|uv| \leq n$
 - $|v| \geq 1$
 - for all $i \geq 0$: $uv^i w \in L$
- In simple terms, this means that if a string v is 'pumped', i.e., if v is inserted any number of times, the resultant string still remains in L .



Process of pumping Lemma

- Suppose that we need to prove that language L is a non-regular.
- Assume that L is a regular language, then L must satisfy pumping lemma property.
- Choose $z \in L$ such that $|z| \leq n$, split z into 3 parts.
 - $|uv| \leq n$
 - $|v| \geq 1$



- If there exist at least one variable for i such that $uv^i w$ not satisfy pumping property so it is a contradiction, therefore language L is non-regular.

L , then L does lemma

Q Proof that language $L = \{a^m b^n \mid m=n\}$ is non-regular?

$L = \{ab, aabb, aaabbb, \dots\} z \in L$

$Z = a^k b^k$

For $i=1 \rightarrow a^{k-1} a b^k$ ✓
 $uv^i w$
 $\rightarrow a^{k-1} a^2 b^k$
 $a^k b^k$ For $i=2 \rightarrow a^{k+1} b^k$
 $uv^i w$
 $\rightarrow a^{k+1} b^k$

b^{k-1}
 w

Q Proof that language $L = \{a^m b^n \mid m < n\}$ is non-regular?

$$z = a^{k-1} b^k$$

$$= \frac{a^{k-2}}{u} \frac{a}{v} \frac{b^k}{w}$$

$$v^{i=1} = v$$

$$v^{i=2} = a^{k-2} a^2 b^k$$

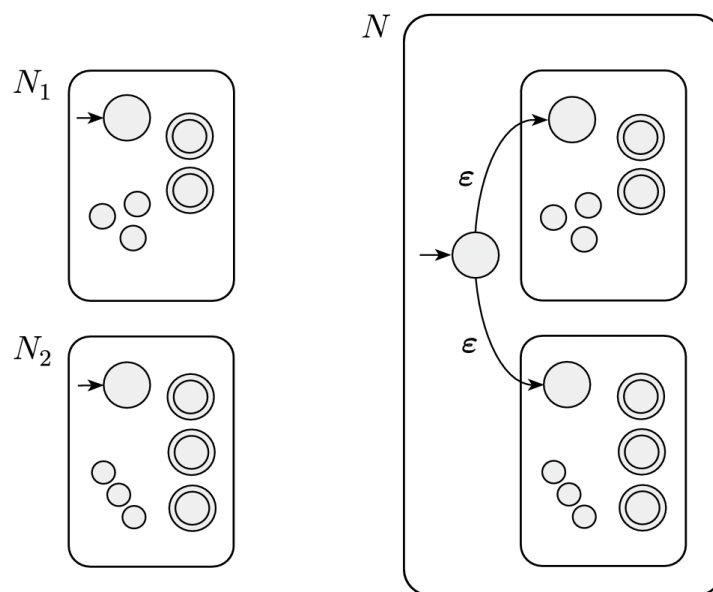
$$= a^k b^k \notin L$$

The class of regular languages is closed under the union operation.

PROOF IDEA We have regular languages A_1 and A_2 and want to prove that $A_1 \cup A_2$ is regular. The idea is to take two NFAs, N_1 and N_2 for A_1 and A_2 , and combine them into one new NFA, N .

Machine N must accept its input if either N_1 or N_2 accepts this input. The new machine has a new start state that branches to the start states of the old machines with ε arrows. In this way, the new machine nondeterministically guesses which of the two machines accepts the input. If one of them accepts the input, N will accept it, too.

We represent this construction in the following figure. On the left, we indicate the start and accept states of machines N_1 and N_2 with large circles and some additional states with small circles. On the right, we show how to combine N_1 and N_2 into N by adding additional transition arrows.



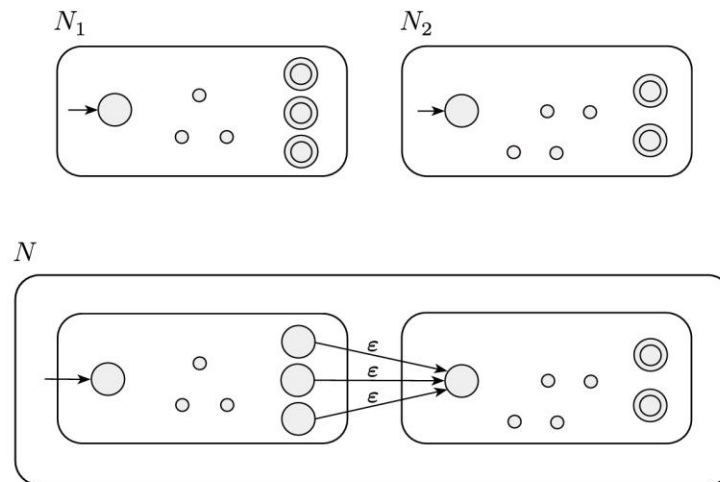
The class of regular languages is closed under the concatenation operation.

PROOF IDEA We have regular languages A_1 and A_2 and want to prove that $A_1 \circ A_2$ is regular. The idea is to take two NFAs, N_1 and N_2 for A_1 and A_2 , and combine them into a new NFA N as we did for the case of union, but this time in a different way, as shown in Figure 1.48.

Assign N 's start state to be the start state of N_1 . The accept states of N_1 have additional ε arrows that nondeterministically allow branching to N_2 whenever N_1 is in an accept state, signifying that it has found an initial piece of the input that constitutes a string in A_1 . The accept states of N are the accept states of N_2 only. Therefore, it accepts when the input can be split into two parts, the first accepted by N_1 and the second by N_2 . We can think of N as nondeterministically guessing where to make the split.

Copyright 2012 Cengage Learning. All Rights Reserved. May not be copied, scanned, or duplicated, in whole or in part. Due to electronic rights, some third party content may be suppressed from the eBook and/or eChapter(s). Editorial review has deemed that any suppressed content does not materially affect the overall learning experience. Cengage Learning reserves the right to remove additional content at any time if subsequent rights restrictions require it.

1.2 NONDETERMINISM 61



THEOREM 1.49

The class of regular languages is closed under the star operation.

PROOF IDEA We have a regular language A_1 and want to prove that A_1^* also is regular. We take an NFA N_1 for A_1 and modify it to recognize A_1^* , as shown in the following figure. The resulting NFA N will accept its input whenever it can be broken into several pieces and N_1 accepts each piece.

We can construct N like N_1 with additional ϵ arrows returning to the start state from the accept states. This way, when processing gets to the end of a piece that N_1 accepts, the machine N has the option of jumping back to the start state to try to read another piece that N_1 accepts. In addition, we must modify N so that it accepts ϵ , which always is a member of A_1^* . One (slightly bad) idea is simply to add the start state to the set of accept states. This approach certainly adds ϵ to the recognized language, but it may also add other, undesired strings. Exercise 1.15 asks for an example of the failure of this idea. The way to fix it is to add a new start state, which also is an accept state, and which has an ϵ arrow to the old start state. This solution has the desired effect of adding ϵ to the language without adding anything else.

